

# Optimierungsverfahren — Blueprint (Teil 2)

Grundlagen · Lineare Opt. & Transport · Analytisch (Hesse, KKT) · QO/SQO · Gradientenfrei · Metaheuristiken · Mehrziel · Methoden-Tabelle — Formeln & Abläufe

## Grundbegriffe & Klassifikation

### Optimierungsproblem

$$\min_{\vec{x}} f(\vec{x}) \text{ u.d.N. } h_k(\vec{x}) = 0, g_j(\vec{x}) \leq 0, x_i^l \leq x_i \leq x_i^u$$

$f$  Zielfunktion,  $h$  Gleichungs-,  $g$  Ungleichungs-Nebenbedingungen (NB), Box-Grenzen. *Maximieren* = *Minimieren* von  $-f$ .

### Klassifikation eines Problems

- **linear / nichtlinear:** sind  $f$  und alle NB linear? (kein  $x^2$ ,  $\sqrt{\cdot}$ ,  $\sin$ ,  $x_i x_j$ )
- **NB-Typen:** ohne / Grenzen / Gleichung / Ungleichung
- **Variablen:** kontinuierlich  $\mathbb{R}^n$  / ganzzahlig  $\mathbb{Z}^n$
- **Ziele:** ein- / mehrkriteriell

### Lokal vs. global

$$\text{lokal: } \arg \min_{x \in N(x_0)} f \quad \text{global: } \arg \min_{x \in \Omega} f$$

**unimodal** = 1 Optimum, **multimodal** = mehrere. Lokaler Algorithmus löst nur unimodal, globaler auch multimodal. *konvex*  $\Rightarrow$  *lokal* = *global*.

## Modellierung — Ablauf

1. Entscheidungsvariablen definieren (mit **Einheit**).
2. Zielfunktion  $f$  aufstellen (was min/max?). *Gewinne/Kosten sind Koeffizienten von  $f$ , keine NB!*
3. pro Ressource/Beschränkung eine NB-Ungleichung.
4. Nichtnegativität  $x_i \geq 0$  & Grenzen.
5. Linearität prüfen.

## Lineare Optimierung

### Standardform (für Solver/Simplex)

- nur **Minimierung**; nur **Gleichungs-NB**
- nur  $x_i \geq 0$ ; rechte Seite  $b_j \geq 0$

$$\min_{\vec{x}} \vec{c}^T \vec{x} \text{ u.d.N. } A\vec{x} = \vec{b}, \vec{x} \geq 0$$

**Umwandlungen:**  $a^T x \leq b \Rightarrow a^T x + s = b$  ( $s \geq 0$ );  $a^T x \geq b \Rightarrow a^T x - s = b$ ; freie Var.  $x \in [-5, 1] \Rightarrow x' = x + 5 \geq 0$ .

### Simplex — Ablauf (geometrisch)

1. Start an einem Eckpunkt des zulässigen Bereichs.
  2. entlang der Kante mit **größter Verbesserung** zum nächsten Eckpunkt.
  3. beenden, wenn keine Verbesserung mehr.
- Optimum liegt stets in einer Ecke; pro Schritt wird ein LGS gelöst (Gauß).*

### Eckpunkte rechnerisch finden

1. NB als *Gleichungen* (auch Achsen  $x_1=0, x_2=0$ ), je zwei schneiden  $\Rightarrow$  Kandidat.
2. nur behalten, wenn *alle* anderen NB erfüllt (zulässig).
3.  $f$  in jedem zul. Eckpunkt  $\Rightarrow$  bester Wert = Optimum.

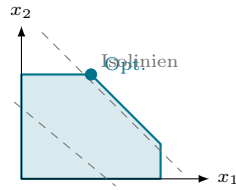
### Ganzzahlige LP ( $\vec{x} \in \mathbb{Z}^n$ )

- **naiv:** kontinuierlich lösen, auf nächste *zulässige* ganze Zahl runden (*nicht immer optimal*)
- **Schnittebenen:** kont. lösen  $\rightarrow$  ganzzahlig? fertig  $\rightarrow$  sonst Schnitt (verbietet die gebrochene Lsg, erlaubt alle ganzen)  $\rightarrow$  wiederholen

## Grafische Lösung (2 Variablen)

### Vorgehen

NB als Geraden zeichnen  $\Rightarrow$  zulässiger Bereich (Schnitt der Halbebenen). Isolinie von  $f$  in Verbesserungsrichtung verschieben, bis sie den Bereich „zuletzt“ berührt  $\Rightarrow$  Optimum (meist Ecke).  $\vec{x}$  ablesen, in  $f$  einsetzen.



### Nichtlinear

Zulässig = Punkte, die *alle* NB erfüllen; bei Gleichungs-NB die **Schnittpunkte** der Kurven *innerhalb* der Grenzen. Davon den mit bestem  $f$ -Wert.

### Slackvariable

$$g_j(\vec{x}) + s_j = b_j, \quad s_j \geq 0$$

**aktiv** (Gerade berührt Optimum)  $\Rightarrow s_j = 0$ ;  
**inaktiv**  $\Rightarrow s_j = b_j - (\text{linke Seite}) > 0$ .

## Klassisches Transportproblem

$m$  Werke (Angebot  $a_i$ )  $\rightarrow$   $n$  Läden (Nachfrage  $b_j$ ), Kosten  $c_{ij}$ , Mengen  $x_{ij} \geq 0$ .

- **Kosten** =  $\sum_{i,j} c_{ij} x_{ij}$
- **zulässig:** jede Zeilensumme =  $a_i$ , jede Spaltensumme =  $b_j$
- **balanciert:**  $\sum a_i = \sum b_j$  (sonst *Dummy-Zeile/-Spalte*)

### Startlösung — Abläufe

**Nord-West-Ecke (NWER):** nordwestlichste freie Zelle mit min(Rest-Angebot, Rest-Nachfrage) füllen, beides verringern, erschöpfte Zeile/Spalte mit 0, wiederholen. *ignoriert Kosten.*

**Matrixminimum (MMM):** immer die freie Zelle mit *geringsten Kosten* zuerst füllen.

**Vogel (VAM):** pro Zeile/Spalte Opportunitätskosten = (zweitkleinste – kleinste Kosten); höchsten Wert wählen, dort günstigste Zelle füllen; wiederholen.

### Optimieren

Startlösung verbessern mit **Stepping-Stone** oder **MODI**.

## Analytische Optimierung (unrestringiert)

### Notwendig

$$\nabla f(\vec{x}) = \vec{0} \Rightarrow \text{stationäre Punkte (Koordinaten lösen)}$$

### Hinreichend — Hesse-Matrix

$$H = \nabla^2 f = \begin{pmatrix} f_{x_1 x_1} & f_{x_1 x_2} \\ f_{x_2 x_1} & f_{x_2 x_2} \end{pmatrix} \quad (2. \text{ Ableitungen!})$$

- $H$  positiv def.  $\Rightarrow$  **Minimum**; neg. definit  $\Rightarrow$  **Maximum**
- $H$  indefinit  $\Rightarrow$  **Sattelpunkt**; semidefinit  $\Rightarrow$  unklar

$2 \times 2$ : pos. def.  $\Leftrightarrow H_{11} > 0 \wedge \det H > 0$ ; neg. def.  $\Leftrightarrow H_{11} < 0 \wedge \det H > 0$ ; indef.  $\Leftrightarrow \det H < 0$ .

**Über Eigenwerte:** alle  $\lambda > 0$  pos.def.; alle  $\lambda < 0$  neg.def.; gemischte Vorzeichen  $\Rightarrow$  indefinit (Sattel).

## Gradientenverfahren

$$\vec{x}_{k+1} = \vec{x}_k - \eta \nabla f(\vec{x}_k), \quad \eta > 0$$

- $\eta$  zu groß  $\rightarrow$  Divergenz; zu klein  $\rightarrow$  langsam
  - findet nur **lokales** Min, braucht Gradient
- Ablauf:  $\nabla f(\vec{x}_k)$  berechnen  $\rightarrow$  Schritt  $\rightarrow$  neuen Wert  $f$  bestimmen  $\rightarrow$  wiederholen.

## Lagrange & KKT

$$\min f \text{ u.d.N. } h(\vec{x}) = 0 \text{ (Mult. } a, \text{ frei)}, g(\vec{x}) \leq 0 \text{ (Mult. } b):$$

$$L(\vec{x}, a, b) = f(\vec{x}) + a h(\vec{x}) + b g(\vec{x})$$

**KKT-Bedingungen (alle 4!):**

1. **Stationarität:**  $\nabla_{\vec{x}} L = \vec{0}$  (je  $x_i$  ableiten = 0)
2. **Zulässigkeit:**  $h = 0, g \leq 0$
3. **Komplementärschlupf:**  $b g(\vec{x}) = 0$
4. **Vorzeichen:**  $b \geq 0$

**Lösen:** Komplementärschlupf  $\Rightarrow$  Fallunterscheidung

- $b = 0$  (NB inaktiv)    •  $g(\vec{x}) = 0$  (NB aktiv).
- Jeden Fall lösen, dann  $g \leq 0$  und  $b \geq 0$  prüfen. Min/Max über Hesse von  $L$ .

## Quadratische Optimierung (QO)

Quadratische Zielfunktion, **lineare** NB:

$$\min_{\vec{x}} \frac{1}{2} \vec{x}^T Q \vec{x} + \vec{c}^T \vec{x} \text{ u.d.N. } A\vec{x} \leq \vec{b}, A_{eq} \vec{x} = \vec{b}_{eq}$$

- $Q$  **symmetrisch** ( $Q = Q^T$ )
  - $Q$  pos. definit  $\Rightarrow$  **konvex**, eindeutige Lösung
- Verfahren: *Innere-Punkte, konjugierter Gradient, Simplex-Varianten, erw. Lagrange.*

## Sequentielle Quadr. Opt. (SQO)

Nichtlineares Problem mit NB: löse iterativ eine **quadratische Näherung**:

$$\min_{\vec{d}} f(\vec{x}_i) + \nabla f(\vec{x}_i)^T \vec{d} + \frac{1}{2} \vec{d}^T H \vec{d}$$

$H$  = Hesse der *Lagrange-Funktion* (oft per *BFGS* genähert).

1. Startlösung  $\vec{x}_0$ .
2. Teilproblem lösen  $\Rightarrow$  Suchrichtung  $\vec{d}$ .
3. Schrittweite  $c > 0$  per Liniensuche.
4.  $\vec{x}_{i+1} = \vec{x}_i + c \vec{d}$ ; konvergiert? sonst zu 2.

*quadr. Näherung von  $f$ , lineare Näherung der NB  $\Rightarrow$  als QO lösbar. lokal, gradientbasiert, deterministisch.*

## Nelder-Mead (Downhill-Simplex)

Gradientenfrei. Simplex aus  $n+1$  Punkten, **sortiert**  $x_0$  best ...  $x_n$  schlecht. Parameter  $\alpha=1, \gamma=2, \beta=0,5$ .

1. **Zentrum** (ohne schlechtesten):  $x_c = \frac{1}{n} \sum_{i \neq n} x_i$
2. **Reflexion:**  $x_r = x_c + \alpha(x_c - x_n)$

**Entscheidung über  $f(x_r)$ :**

- $f(x_r) < f(x_0)$ : **Expansion**  $x_e = x_c + \gamma(x_r - x_c)$ ; besseren von  $x_e, x_r$  nehmen
- $f(x_0) \leq f(x_r) < f(x_{n-1})$ :  $x_r$  **übernehmen**
- $f(x_r) \geq f(x_{n-1})$ : **Kontraktion**  $x_k = x_c + \beta(x_n - x_c)$ ; besser als  $x_n$ ? übernehmen, sonst **Schrumpfen** (alle Richtung  $x_0$ )

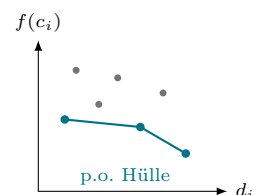
*lokal; global via Neustarts.*

## DIRECT (DIviding RECTangles)

Global, gradientenfrei, deterministisch; nur Box-Grenzen.

1. Init: ein Rechteck (ganzer Raum).
2. **potentiell optimale** Rechtecke bestimmen.
3. **Sample & Divide:** gewähltes Rechteck in größter Dimension dritteln, Mittelpunkte auswerten.
4. wiederholen bis max. Iterationen.

**Potentiell optimal** = Punkte auf der unteren rechten **konvexen Hülle** im Diagramm (Größe  $d_i$  vs. Güte  $f(c_i)$ ): groß *und/oder* gut.



## Evolutionäre Algorithmen (EA)

**Population** von Lösungskandidaten. Pro Generation:

Eltern  $\rightarrow$  Rekombination  $\rightarrow$  Mutation  $\rightarrow$  Bewertung  $\rightarrow$

- **Mutation:** reell  $\vec{x} + \vec{\epsilon}$ ,  $\epsilon \sim N(0, \sigma)$ ; binär: Bit kippen
  - **Rekombination:** Mittelwert  $(\vec{x}_1 + \vec{x}_2)/2$ ; Single-Point-Crossover
  - **Fitness** =  $f(\vec{x})$
- stochastisch, gradientenfrei, lokal & global;  
Hyperparameter: Populationsgröße, Mutationsrate/-schrittweite.

## (1+1)-ES — 1/5-Erfolgsregel

Parameter: Speicherlänge  $g$ , Multiplikator  $a > 1$ , Schrittweite  $\sigma$ .

1. Start  $\vec{x}$ ,  $y = f(\vec{x})$ .
2. Clone  $\vec{x}^{\text{next}} = \vec{x}$ .
3. Mutation  $x_i^{\text{next}} \pm \epsilon_i$ ,  $\epsilon_i \sim N(0, \sigma_i)$ .
4. Auswerten  $y^{\text{next}} = f(\vec{x}^{\text{next}})$ .
5. **Selektion** (min):  $y^{\text{next}} < y$ ? Erfolg (1) speichern,  $\vec{x} = \vec{x}^{\text{next}}$ ; sonst (0).
6. **Rate**  $s = \frac{\text{Erfolge im Fenster (inkl. jetzt)}}{g}$ .
7.  $s > \frac{1}{5} \Rightarrow \sigma \cdot a$ ;  $s < \frac{1}{5} \Rightarrow \sigma/a$ ;  $s = \frac{1}{5} \Rightarrow$  gleich. Zu 2.

Rate ist Anzahl Erfolge /  $g$  — nicht die  $f$ -Differenz!

## Weitere Metaheuristiken

### CMA-ES

Evolutionstrategie mit fortlaufend angepasster **Kovarianzmatrix** (ideale Suchverteilung). Stand der Technik, kontinuierlich; global wenn globale Struktur nutzbar, sonst Neustarts.

### Simulated Annealing

Verschlechterung  $\Delta f$  akzeptiert mit  $P = \exp(-\frac{\Delta f}{T})$

## Eigenschaften der Optimierungsmethoden

| Methode                               | Global | Gradienten-basiert | Neben-bedingungen | Ziele   | Linear    | Ansatz     | Genauigkeit    | Determi-nistisch | Variablen                    |
|---------------------------------------|--------|--------------------|-------------------|---------|-----------|------------|----------------|------------------|------------------------------|
| Stationäre Punkte / Hesse-Matrix      | ✓      | ✓                  | ohne              | 1       | ×         | analytisch | exakt          | ✓                | $\mathbb{R}^n$               |
| Gradientenverfahren                   | ×      | ✓                  | ohne              | 1       | ×         | numerisch  | ?              | ✓                | $\mathbb{R}^n$               |
| Simplex (lineare Optimierung)         | ×      | ✓                  | alle              | 1       | ✓         | numerisch  | exakt          | ✓                | $\mathbb{R}^n, \mathbb{Z}^n$ |
| Quadratische Optimierung              | ×      | ✓                  | alle              | 1       | teilweise | numerisch  | ?              | ✓                | $\mathbb{R}^n$               |
| Karush-Kuhn-Tucker                    | ✓      | ✓                  | alle              | 1       | ×         | analytisch | exakt          | ✓                | $\mathbb{R}^n$               |
| Sequentielle quadratische Optimierung | ×      | ✓                  | alle              | 1       | ×         | numerisch  | ?              | ✓                | $\mathbb{R}^n$               |
| Nelder-Mead                           | ×      | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ✓                | $\mathbb{R}^n$               |
| DIRECT                                | ✓      | ×                  | Grenzen           | 1       | ×         | numerisch  | näherungsweise | ✓                | $\mathbb{R}^n$               |
| Evolutionäre Algorithmen              | ✓ (?)  | ×                  | /                 | /       | ×         | numerisch  | näherungsweise | ×                | /                            |
| (1+1)-Evolutionsstrategie             | ×      | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ×                | $\mathbb{R}^n$               |
| CMA-ES                                | ✓ (?)  | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ×                | $\mathbb{R}^n$               |
| Simulated Annealing                   | ✓      | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ×                | $\mathbb{R}^n, \mathbb{Z}^n$ |
| Particle Swarm Optimization           | ✓      | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ×                | $\mathbb{R}^n$               |
| Ant Colony Optimization               | ✓      | ×                  | ohne              | 1       | ×         | numerisch  | näherungsweise | ×                | $\mathbb{Z}^n$               |
| Surrogatmodell-basierte Optimierung   | ✓ (?)  | ×                  | /                 | /       | ×         | numerisch  | näherungsweise | /                | /                            |
| NSGA-II                               | ✓      | ×                  | ohne              | mehrere | ×         | numerisch  | näherungsweise | ×                | $\mathbb{R}^n$               |

✓ = ja    × = nein    (?) = abhängig von Problem und Parametern    „teilweise“ = linear nur in den Nebenbedingungen    „/“ = nicht festgelegt

Hohe  $T$ : viele Sprünge;  $T \downarrow$  über Zeit. Ablauf: Nachbar vorschlagen  $\rightarrow$  besser? akzeptieren  $\rightarrow$  sonst mit  $P$ . erlaubt Verlassen lokaler Optima.

### Particle Swarm (PSO)

Schwarm; jedes Teilchen = Lösung; gute Lösungen beeinflussen Bewegung der schlechteren.

### Ant Colony (ACO)

Ameisen legen **Pheromone**; stärkere Spuren wahrscheinlicher; kurze Wege  $\rightarrow$  mehr Pheromon. diskret/Routing.

### SMBO (Surrogatmodell)

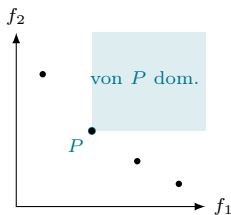
Für **teure**  $f$ : durch günstiges ML-Surrogat ersetzen.

1. Startpunkte echt auswerten (Versuchsplanung).
2. Surrogat anpassen.
3. auf Surrogat besten nächsten Punkt wählen.
4. dort echt auswerten, Surrogat updaten. Zu 2.

## Mehrzieloptimierung

### Dominanz (alle Ziele min)

$P$  **dominiert**  $Q$ :  $P \leq Q$  in *allen* Zielen und  $<$  in mind. einem. **Pareto-optimal** = von keinem dominiert. **Pareto-Menge** (im Variablenraum)  $\rightarrow$  Bild = **Pareto-Front**.



### Non-Dominated-Sorting-Rang

1. Rang 1 = alle nicht dominierten Punkte.
  2. streichen; im Rest erneut nicht-dominierte = Rang 2.
  3. wiederholen, bis alle einen Rang haben.
- 2 Ziele:  $P$  ist Rang 1, wenn kein Punkt links-unten von ihm liegt.

### Crowding Distance

Pro **Zielfunktion** getrennt:

1. nach diesem Ziel sortieren.
2. Randpunkte (min/max):  $CD = \infty$ .
3. innere  $i$ :  $CD += \frac{f_{i+1} - f_{i-1}}{f_{\max} - f_{\min}}$

über alle Ziele **summieren**; große CD bevorzugt (Diversität).

### NSGA-II

EA für mehrere Ziele; Auswahl per **Rang**, bei Gleichstand **Crowding Distance**: Population  $\rightarrow$  Eltern (Rang/CD)  $\rightarrow$  Rekomb.  $\rightarrow$  Mutation  $\rightarrow$  Überlebende (Rang/CD)  $\rightarrow$  wiederholen. SMS-EMOA nutzt stattdessen den **Hypervolumenbeitrag**.

### Hypervolumen

Fläche/Volumen zwischen Front und **Referenzpunkt** (rechts-oben); größer = besser. **Beitrag** eines Punktes = Fläche, die *nur* er abdeckt. Randpunkte oft  $\infty$ .